CS3300 A: Introduction to Software Engineering

Lecture 14: Software Testing; Black-Box Testing

Dr. Nimisha Roy ▸ nroy9@gatech.edu

Slides adapted from Alessandro Orso

Verification and Validation…



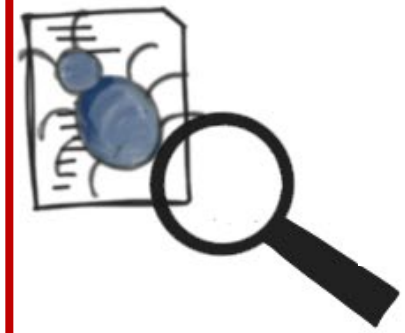
Requirements Engineering



Design



Implementation



Verification & Validation



Maintenance

Why Verification Matters in the era of AI ?

Code	As code generation becomes easier, engineers must be able to:
Define	Define expected software behavior.
Design	Design meaningful tests.
Identify	Identify incorrect assumptions and missing edge cases.
Evaluate	Evaluate AI-generated code and tests.
Assess	Assess whether the implementation satisfies requirements.

Pre-AI: Testing pays off in ~3 weeks

Post-AI: Testing pays off in ~3 days

-Titus Winters (senior principal scientist at Adobe focusing on developer experience)
Based on survey from Industry leaders, October, 2026

The Changing Role of Software Engineers

01

AI tools are increasingly used for implementation, debugging, and code comprehension.

02

Google has reportedly piloted AI-assisted coding and code-comprehension interviews.

03

Evaluating AI-assisted work requires technical understanding, debugging, and verification skills.

Google replaced one traditional coding round with a Code Comprehension round in its May 2026 pilot. Instead of solving a LeetCode-style algorithm from scratch, you receive a 200-500 line unfamiliar codebase in a CoderPad environment, and you have 60 minutes to read it, find bugs, discuss design decisions, and suggest improvements, all while using Google's own Gemini as an AI assistant. If you have been practicing only algorithmic problem-solving, this round will catch you off guard. Here is exactly what the round tests, why it exists, and how to build the skills it requires before your interview day.

https://www.linkedin.com/posts/ericcheng26_google-is-shifting-away-from-leetcode-style-share-7515118002262966272-mQoo/

Software is Buggy!

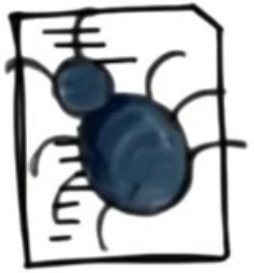
- Cost of bugs: $ 60 B/year
- On average, 1-5 errors per 1K LOC
- Windows 10
  - 50M LOC
  - 63,000 known bugs at the time of release
  - 1.25 per 1,000 lines

- For mass market software 100% correct SW development is infeasible, but
- We must verify the SW as much as possible

Failure, Fault, Error



Failure: Observable incorrect behavior of a program. Conceptually related to the behavior of the program, rather than its code.



Fault (bug): Related to the code. Necessary (not sufficient!) condition for the occurrence of a failure.



Error: Cause of a fault. Usually a human error (conceptual, typo, etc.)

Failure, Fault, Error: Example



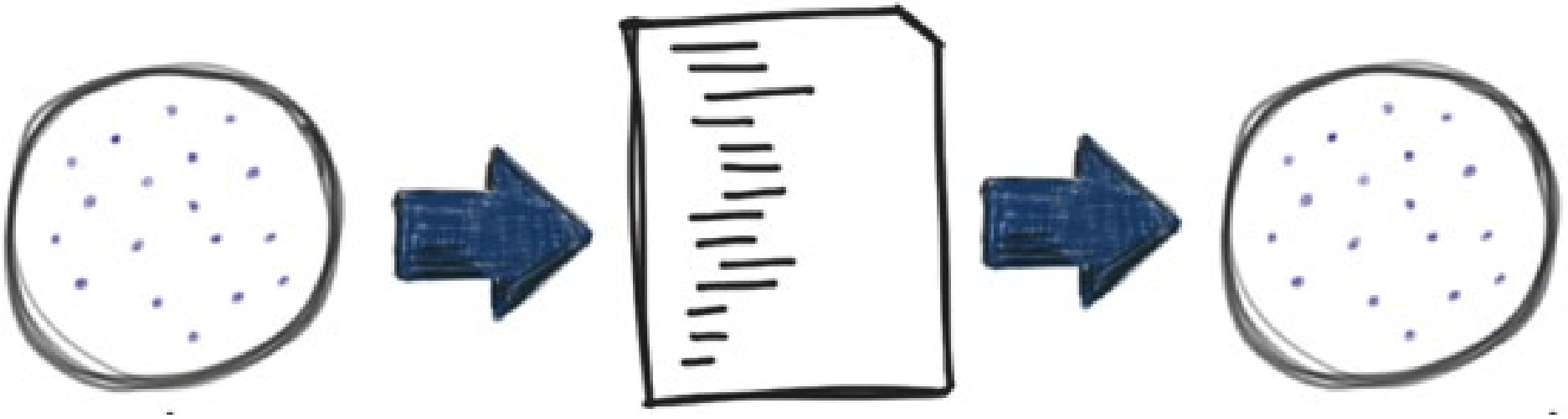
```
1.  double doubleValue(int param) {
2.      double result;
3.      result = (double) param * param;
4.      return(result);
5.  }
```

A call to double(3) returns 9. What is this?	The result 9 is a failure- it is an observable behavior
Where is the fault?	Line 3
What is the error that caused the fault?	N/A. Maybe typo, erroneous copy paste, or conceptual. Only the developer knows.

Approaches to Verification

- **Testing** (dynamic verification): exercising software to try and generate failures
- **Static analysis**: identify (specific) problems statically, that is, considering all possible executions
- **Inspections/reviews/walkthroughs**: systematic group review of program text to detect faults
- **Formal verification** (proof of correctness): proving that the program implements the program specification

Testing



Input Domain D

Software

Output Domain O

Test Case: {i ∈ D, o ∈ O}

Test Suite: A set of Test Cases

Static Verification



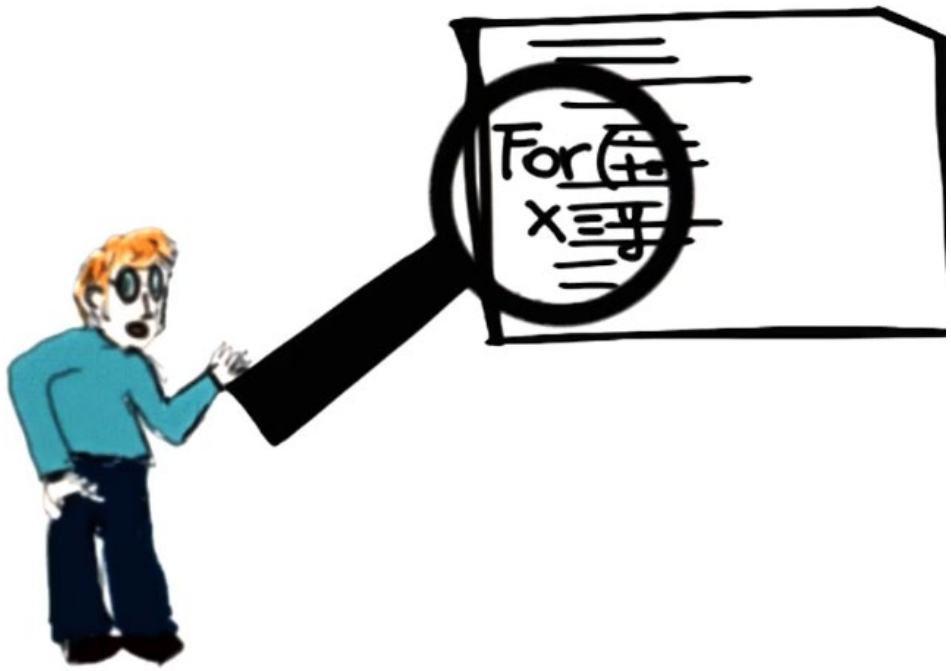
Input Domain D

Software

Output Domain O

Considers all possible inputs (execution/behaviors)

Inspections/Reviews/Walkthroughs



Human intensive activity
Manual
Group activity
Inspect defects in the artifacts by identifying faults

Formal Proof (Of correctness)



Program

Specification

Given a formal specification, checks that the code corresponds to such specification

Sophisticated mathematical analysis

Comparison among the 4 techniques



PROS



CONS

Testing	No False Positives	Highly Incomplete
Static Verification	Considers all program behaviors, Complete	False Positives, Expensive
Inspections	Systematic, Thorough	Informal, Subjective
Formal Proofs of Correctness	Strong Guarantees	Complex, Expensive to build/prove a mathematical basis

What is Testing?

Testing == To execute a program with a sample of the input data

- Dynamic technique: program must be executed
- Optimistic approximation:
  - The program under test is exercised with a (very small) subset of all the possible input data
  - We **assume** that the behavior with any other input is consistent with the behavior shown for the selected subset of input data

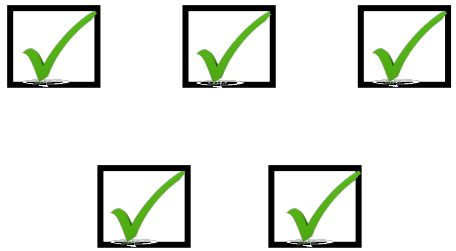
Successful Tests



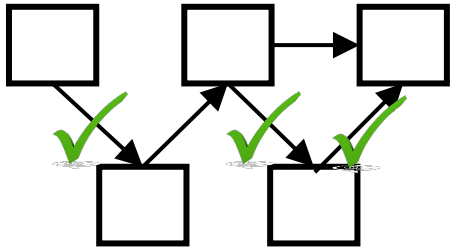
-Goodenough and Gerhart (1985). “Towards a Theory of Test data selection”. *IEEE Transactions of Software Engineering*, Jan 1985

Testing Granularity Levels

Unit Testing

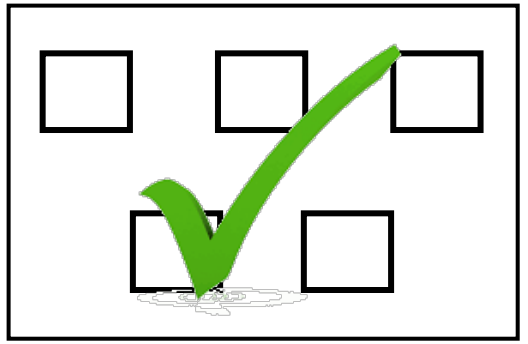


Integration



Big Bang

System

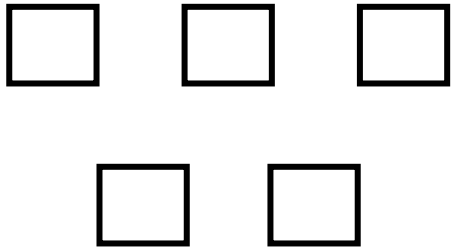


Functional/Non-functional

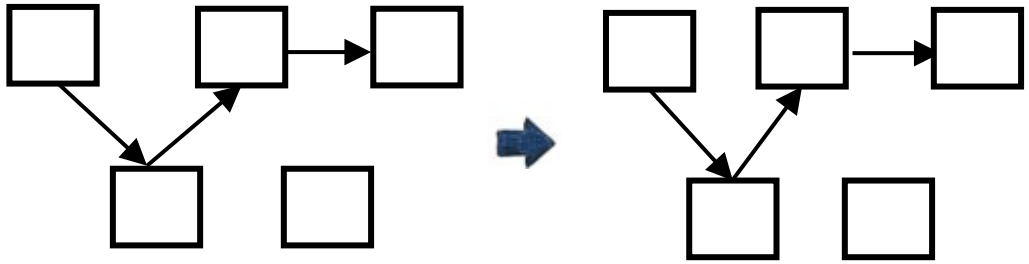


Customer

Acceptance Testing



Regression Testing



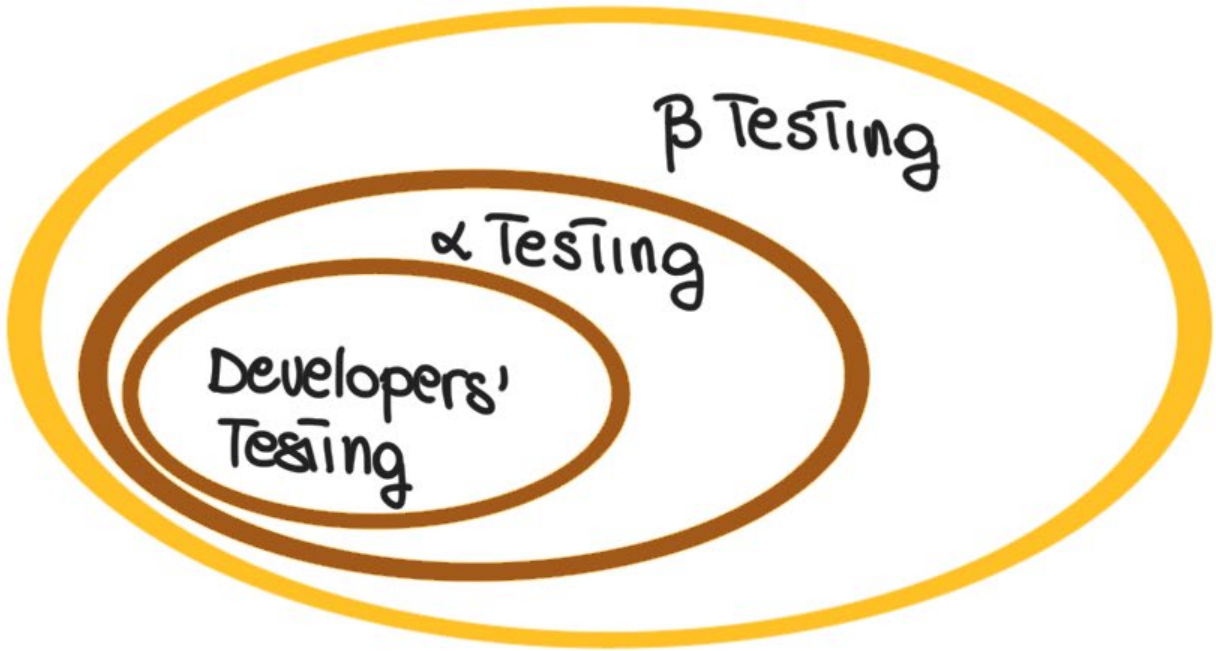
Testing Stages



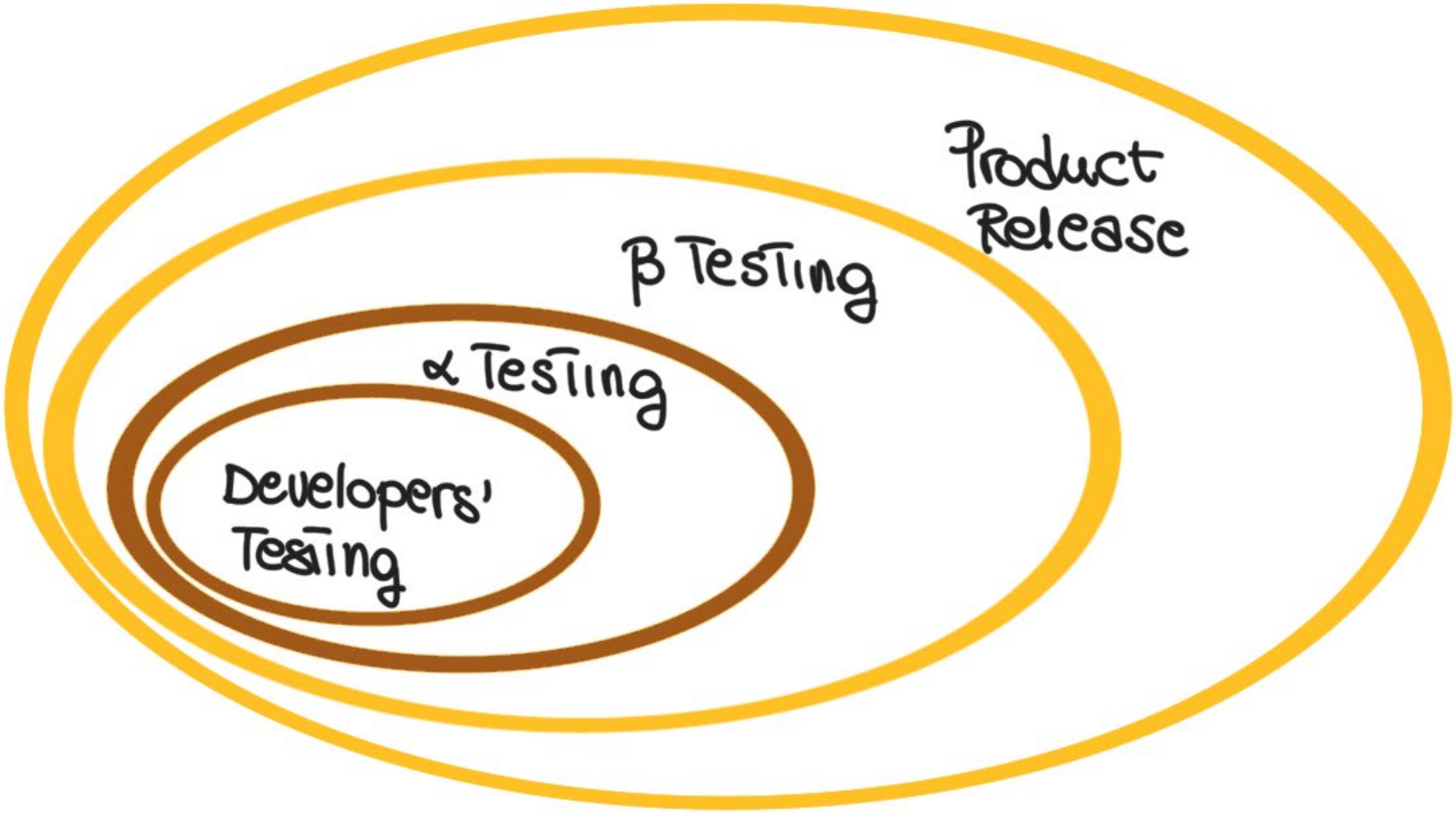
Testing Stages



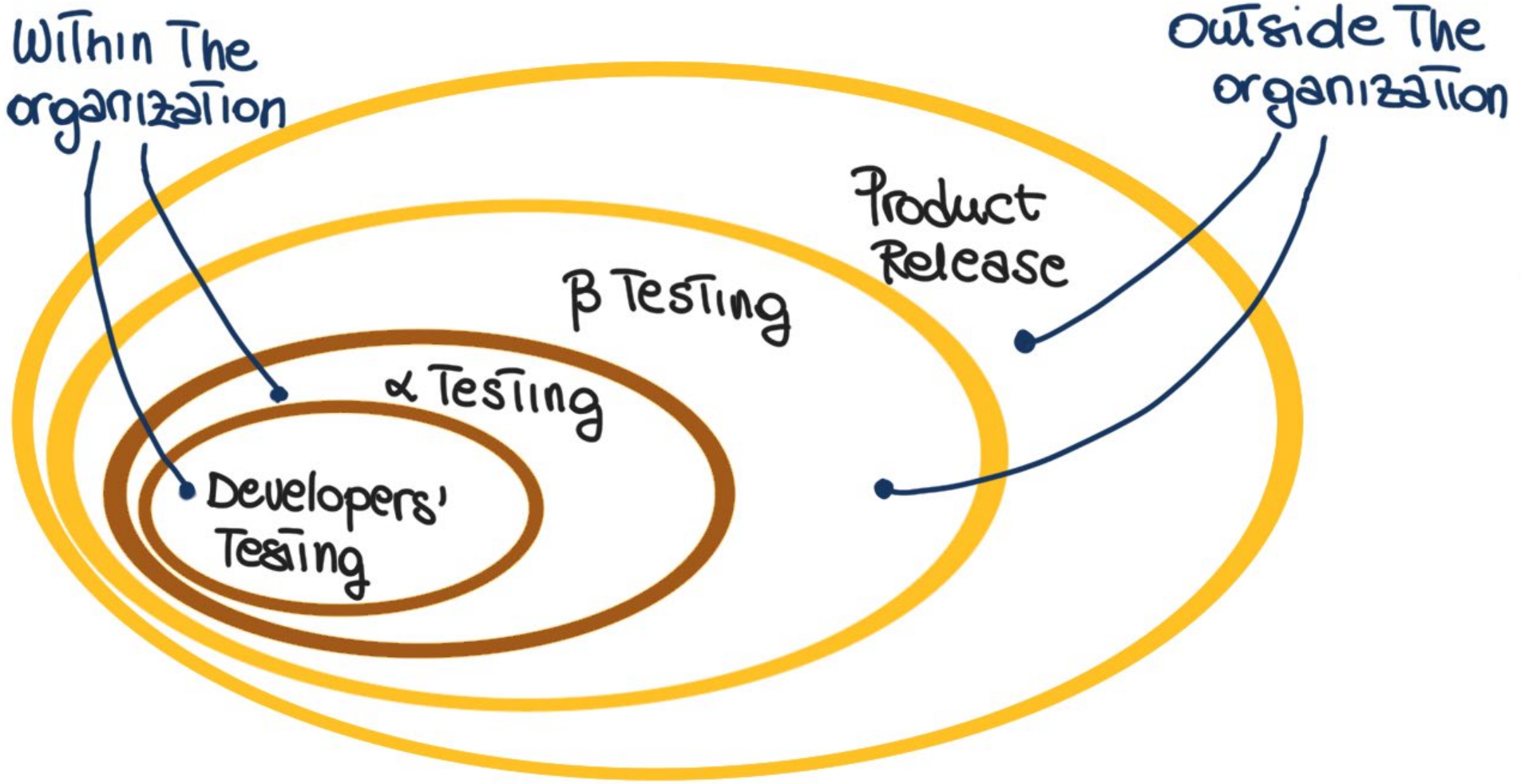
Testing Stages



Testing Stages



Testing Stages



Testing Techniques

There are several techniques

- Different processes
- Different artifacts
- Different approaches

There are no perfect techniques

- Testing is a best-effort activity

There is no best technique

- Different contexts
- Complementary strengths and weaknesses
- Trade-offs

Testing Techniques





BLACK BOX TESTING	WHITE BOX TESTING
Tests externally observable behavior	Tests based on internal code structure
Derived from specifications and requirements	Derived from statements, branches, conditions, and paths
Implementation knowledge not required	Requires implementation knowledge
Examples: equivalence partitions, boundary values	Examples: statement and branch coverage

Black-Box vs White box: Same Program, Different Tests

Specification: Display an integer byte count; values of 1024 or more should be displayed in KB.

Black-box

Tests derived from the specification.

- Positive
- Negative
- Zero
- Threshold values, if identified

```
void printNumBytes(int param) {
    if (param < 1024)
        printf("%d", param);
    else
        printf("%d KB", param / 124);
}
```

Black-Box vs White box: Same Program, Different Tests

Specification: Display an integer byte count; values of 1024 or more should be displayed in KB.

White-box

Tests derived from the code.

- param < 1024
- param >= 1024

Exercise both branches

```
void printNumBytes(int param) {
    if (param < 1024)
        printf("%d", param);
    else
        printf("%d KB", param / 124);
}
```

Which tests are likely to expose the incorrect divisor (124 instead of 1024)?

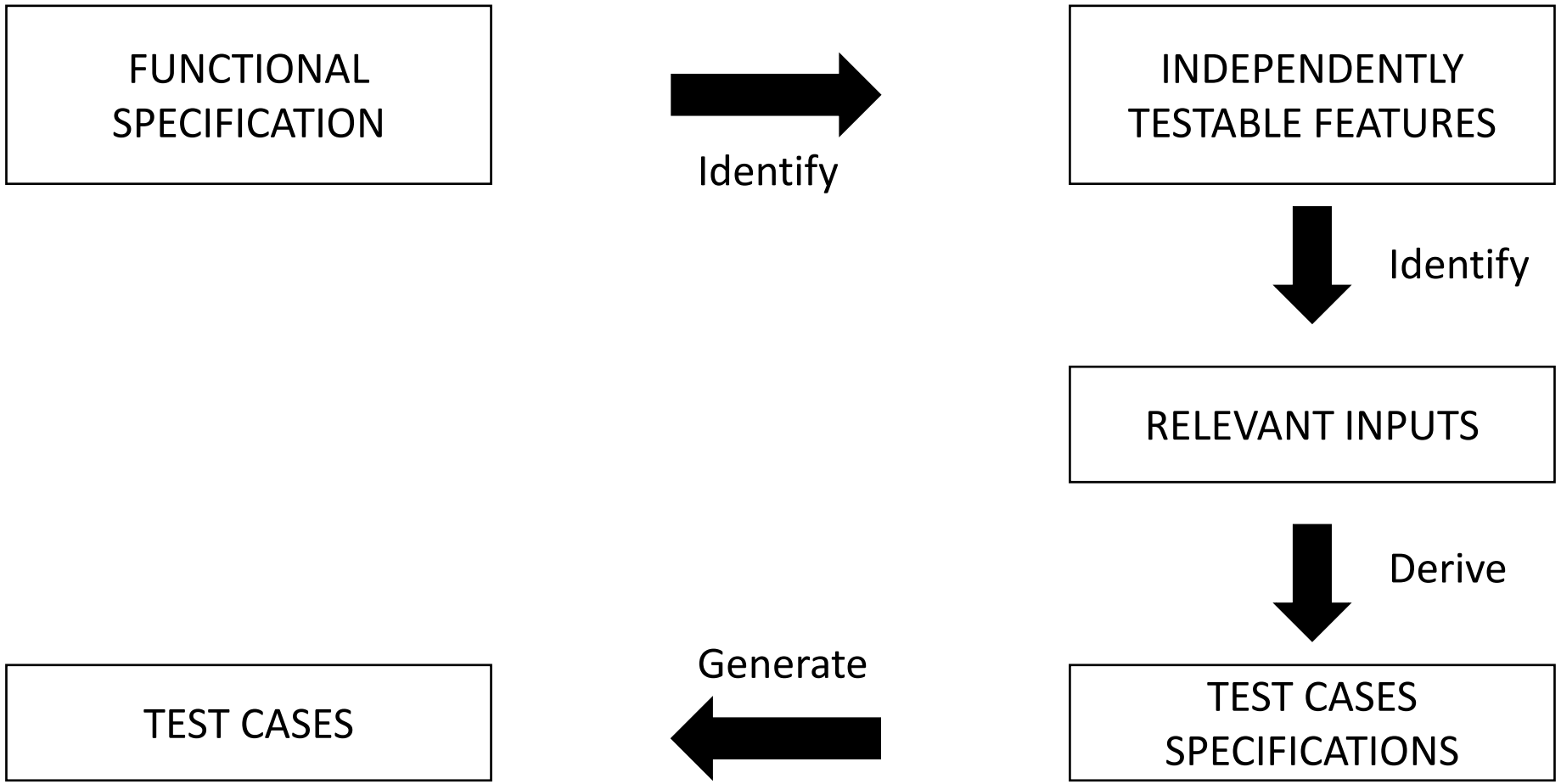
Black Box Testing - From Specifications to Test Cases

FUNCTIONAL SPECIFICATION



TEST CASES

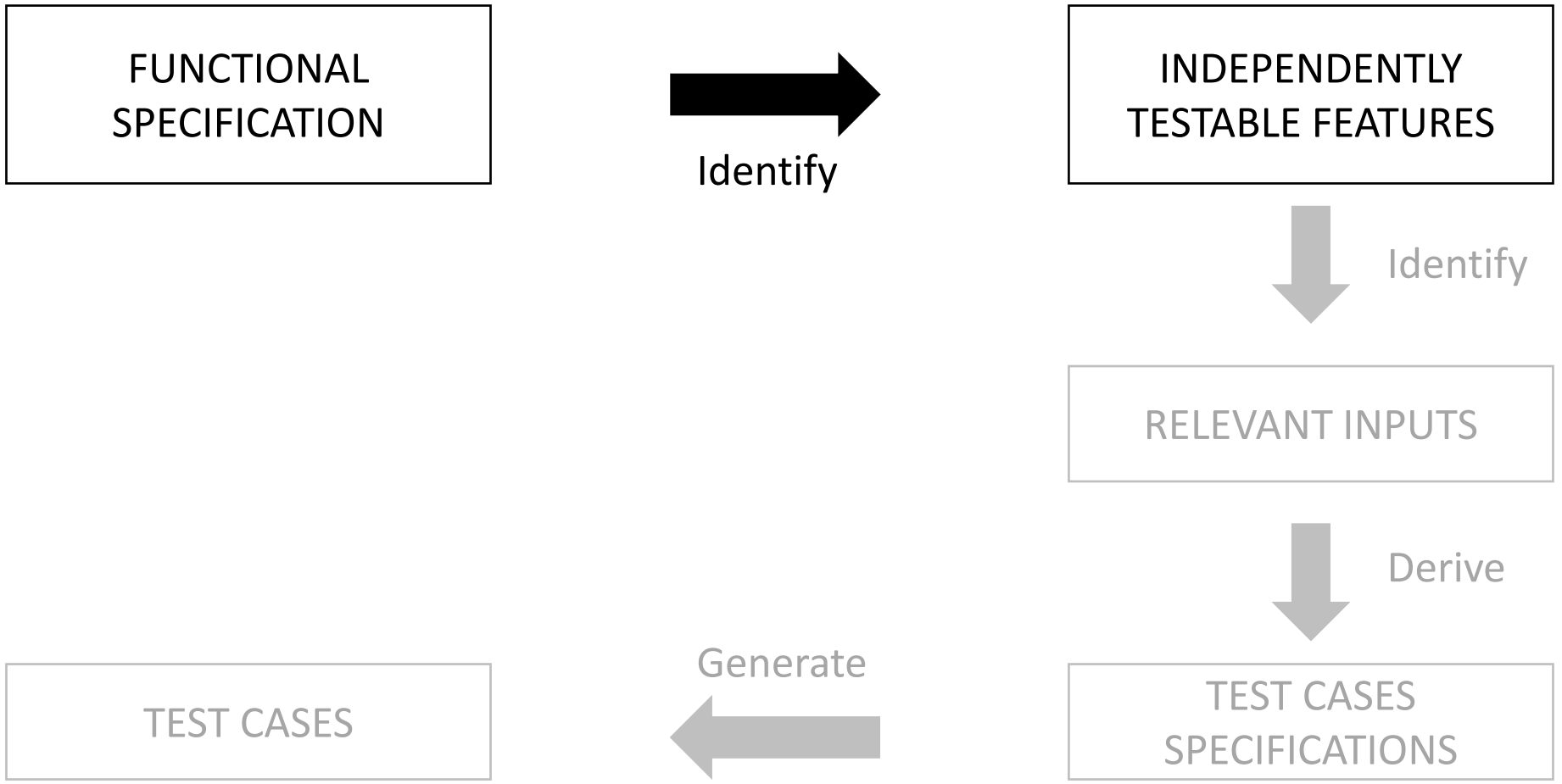
A systematic Functional-Testing Approach



Decoupling; Automated Sub-tasks; Monitor testing process

Slide adapted from Alessandro Orso

A systematic Functional-Testing Approach



Slide adapted from Alessandro Orso

Identifying Testable Features



printSum (int a, int b)

How many independently testable features do we have here?

[✓] 1

[ ] 2

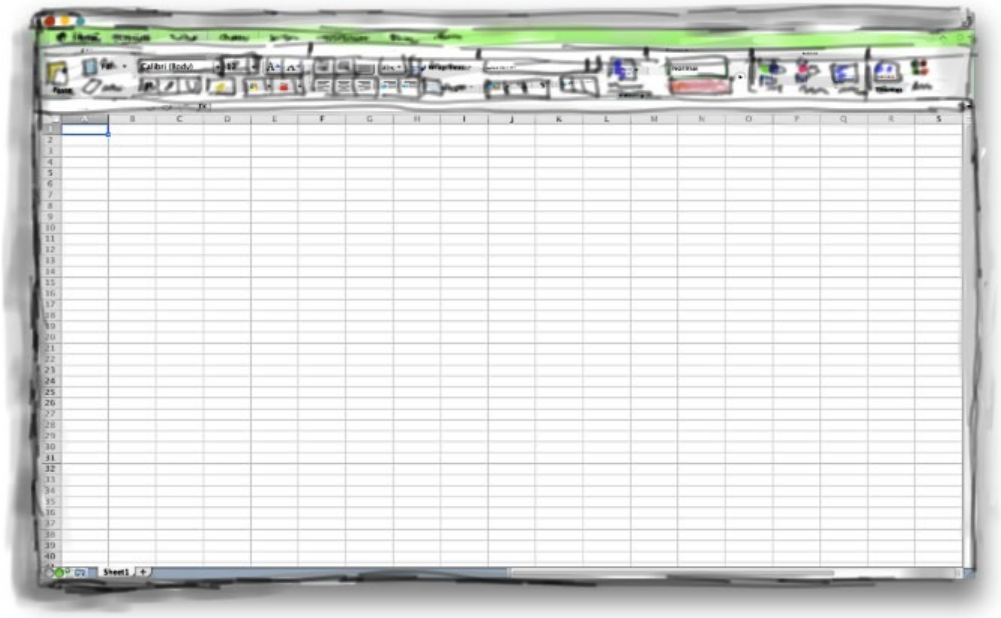
[ ] 3

[ ] 4

Identifying Testable Features



Identify 3 possible independently testable features for a spreadsheet

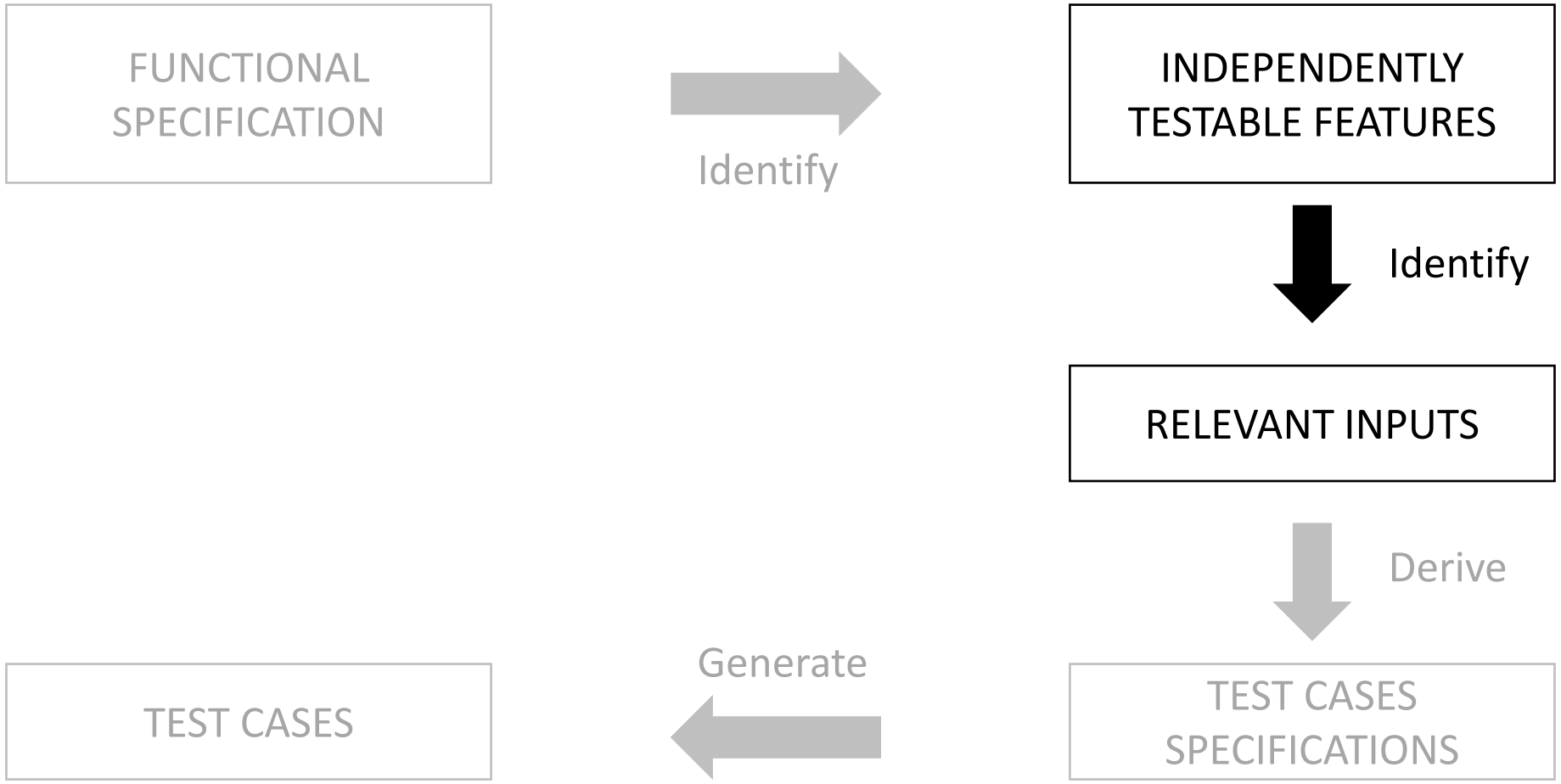


[Statistical Functions]

[Cell Merging]

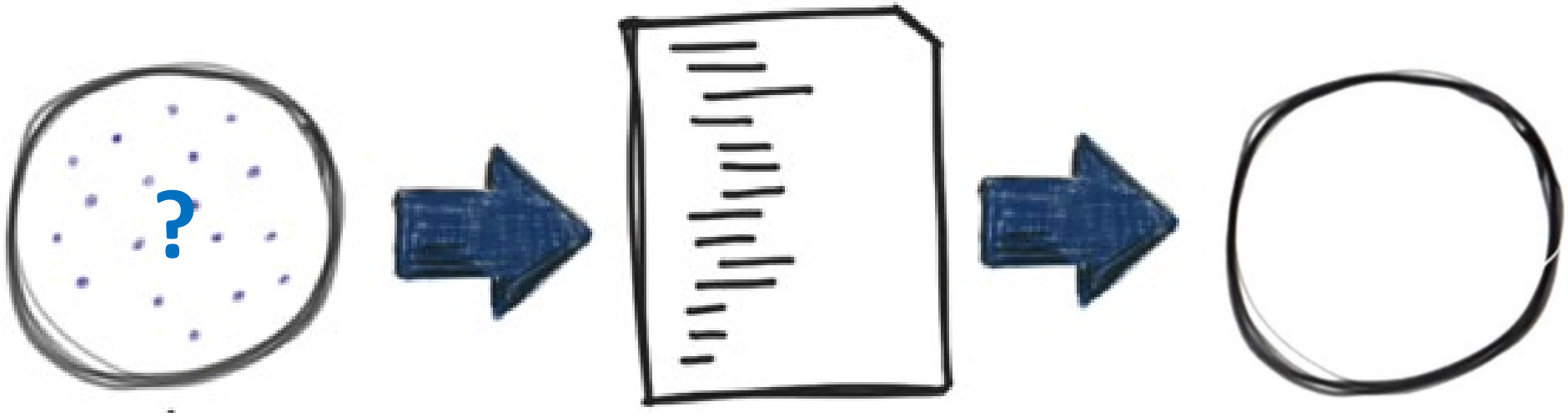
[Chart creation]

A systematic Functional-Testing Approach



Slide adapted from Alessandro Orso

Test Data Selection



Input Domain D

Software

Output Domain O

How to select meaningful set of inputs and corresponding outputs?

Powerful machines, why not exhaustive search?

Straw-Man Idea: Exhaustive Testing!





How long would it take to exhaustively test the function printSum(int a, int b)?

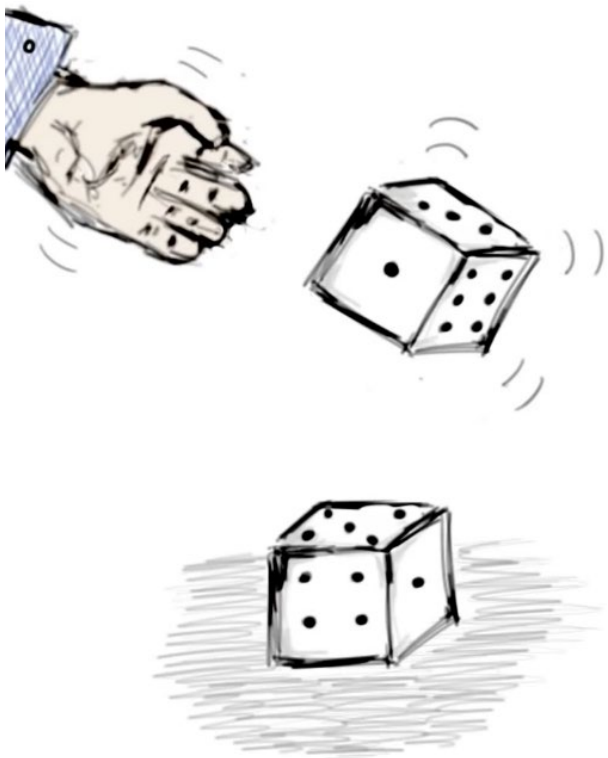
$2^{32} * 2^{32} = 2^{64} \sim= 10^{19}$ tests

1 test per nanosecond

10^9 tests per second

10^{10} seconds overall ~ 600 years

Random Testing



Advantages

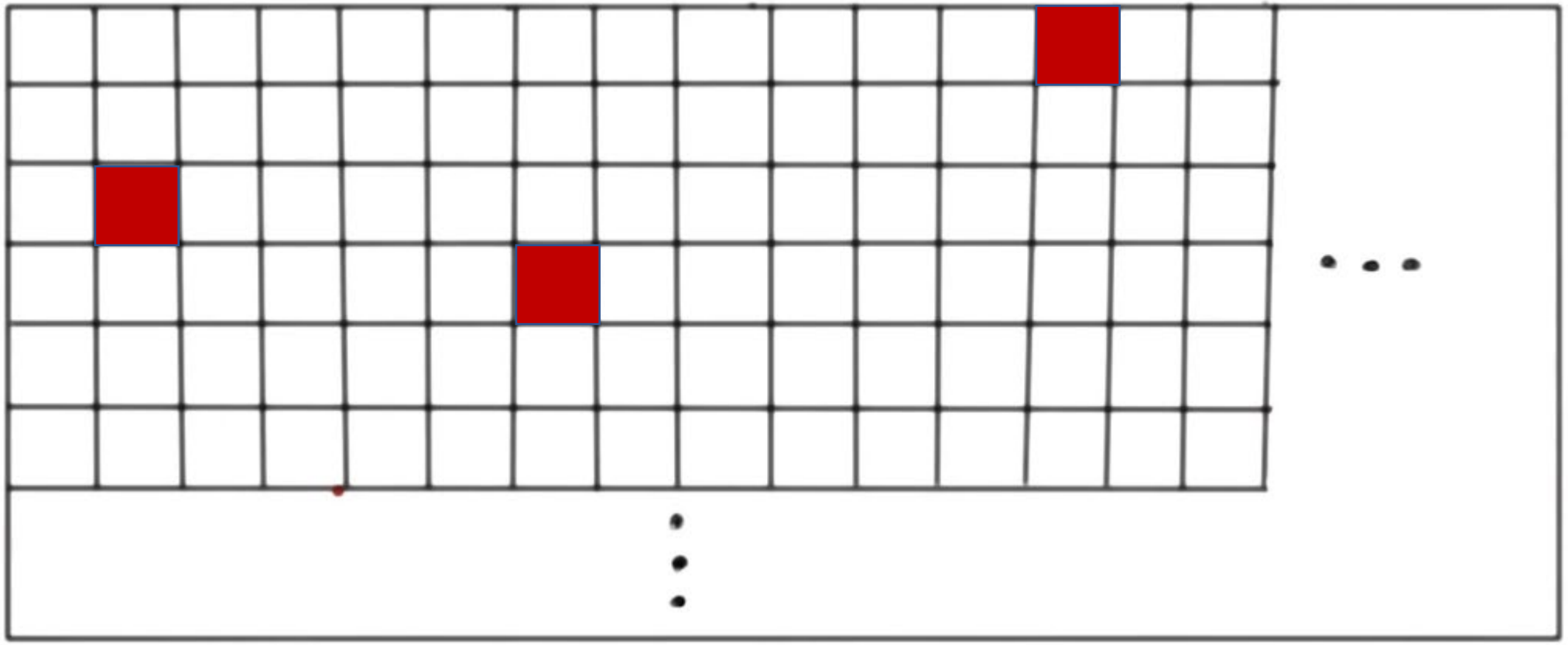
- Pick inputs uniformly
- All inputs considered equal
- No designer bias (developer may develop code based on an assumption, test cases may also be biased)

So why not random?

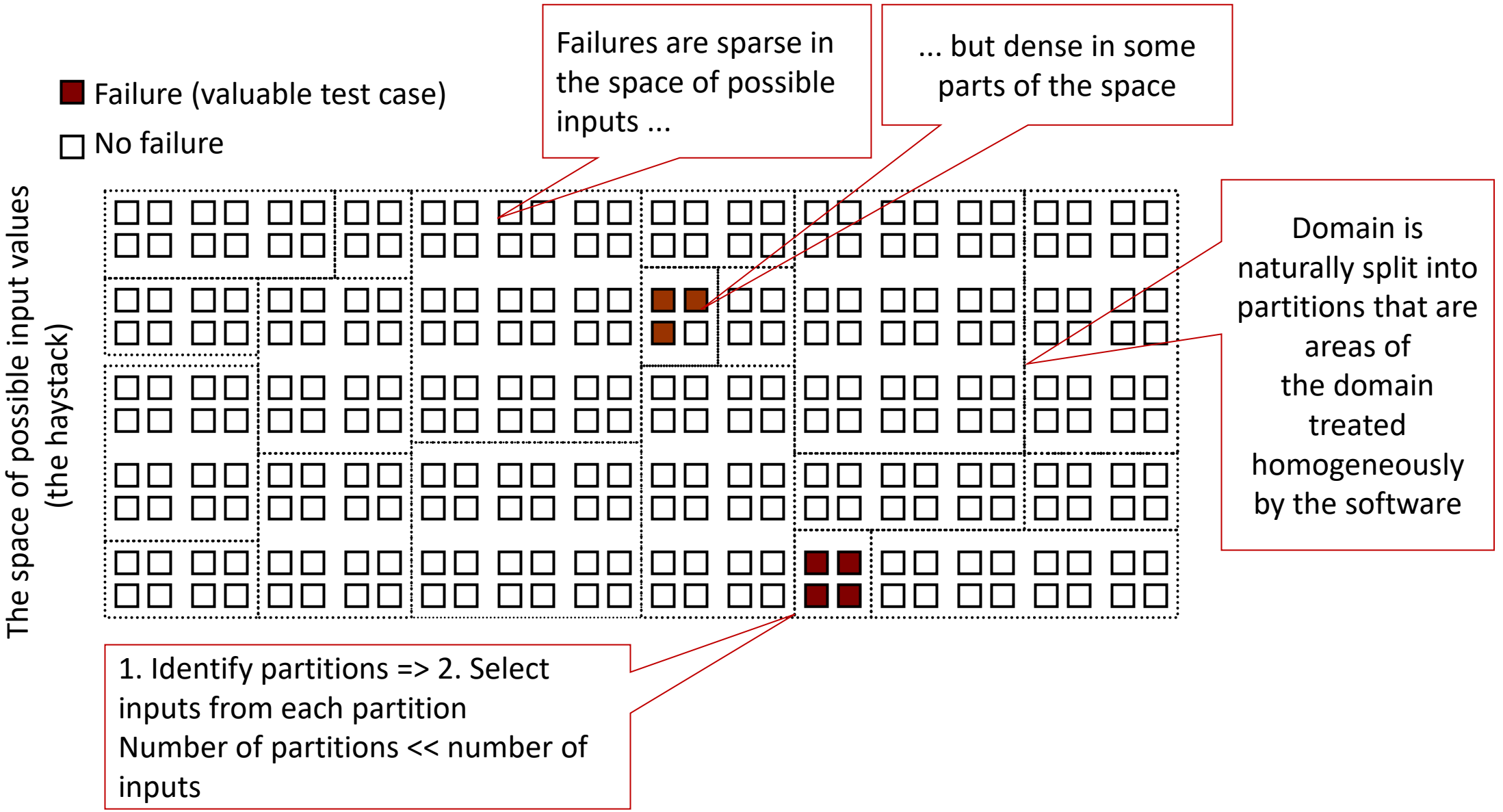


Same as finding many needles in a haystack

So why not random?



Systematic Partition Testing



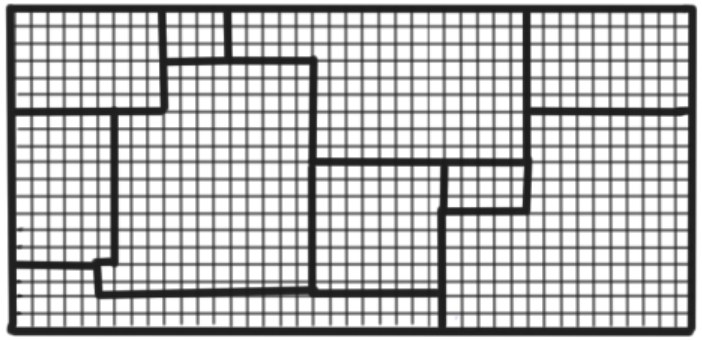
Example

split (string Str, int Size)

1. Identify partitions:

- Size < 0 (Designer bias might let you not pick this partition)
- Size = 0
- Size > 0
- Str with length 0
- Str with length < Size
- Str with length in [Size,Size*2]
- Str with length > Size*2
- …

Boundary Values



2. Select **interesting** Inputs from each partition

Basic Idea: Errors tend to occur at the boundary of a sub-domain

=> Select inputs at these boundaries

Select interesting Inputs from each partition: Example

split (string Str, int Size)

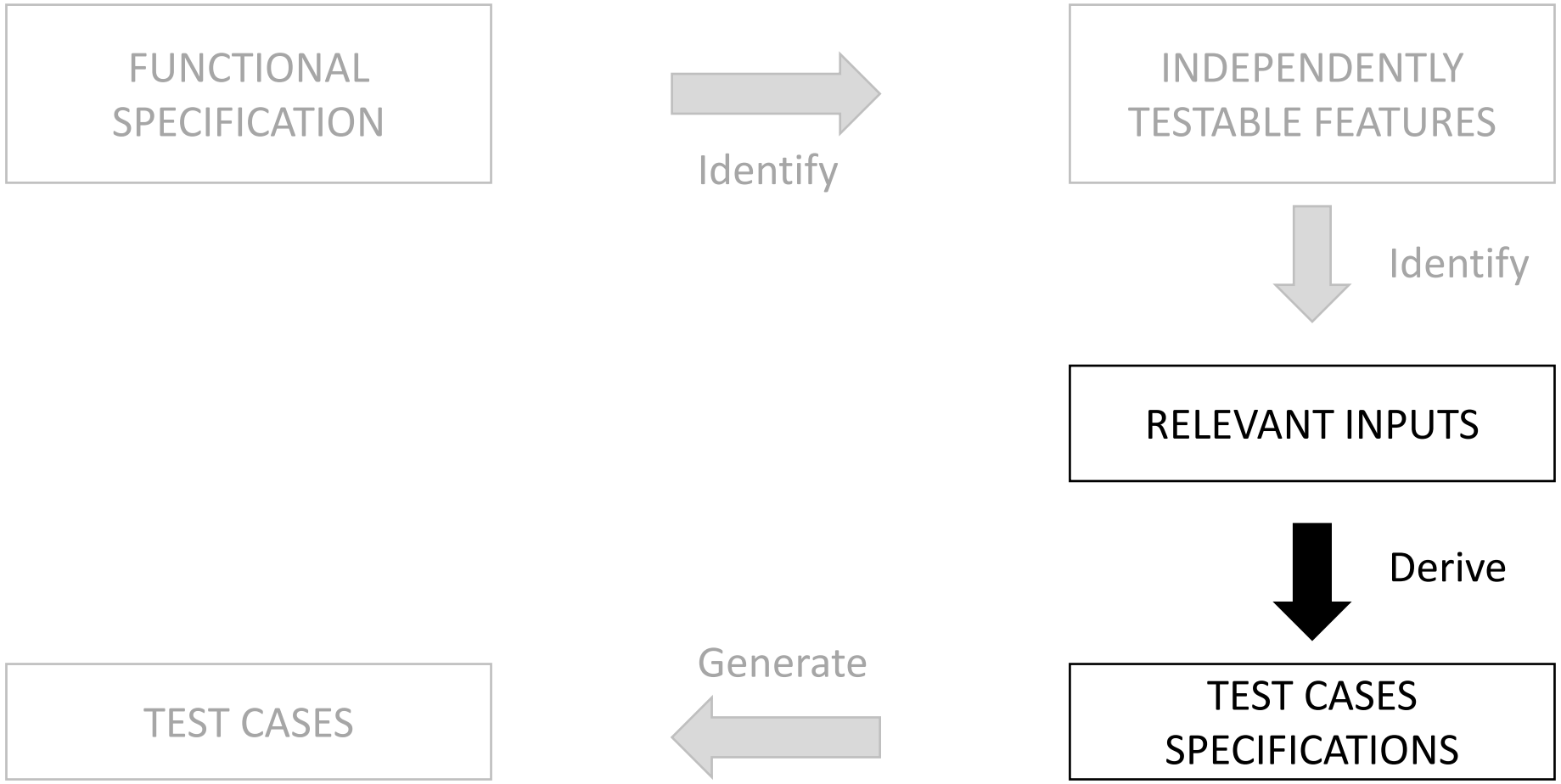
Some possible partitions:

- Size < 0
- Size = 0
- Size > 0
- Str with length < Size
- Str with length in [Size, Size*2]
- Str with length > Size*2

Some possible inputs:

- Size = -1
- Size = 1
- Size = MAXINT
- Str with length = Size- 1
- Str with length = Size
- …

A systematic Functional-Testing Approach



3. Generate Test Case Specifications: Example

split (string Str, int Size)

Some possible inputs:

- Size = -1
- Size = 1
- Size = MAXINT

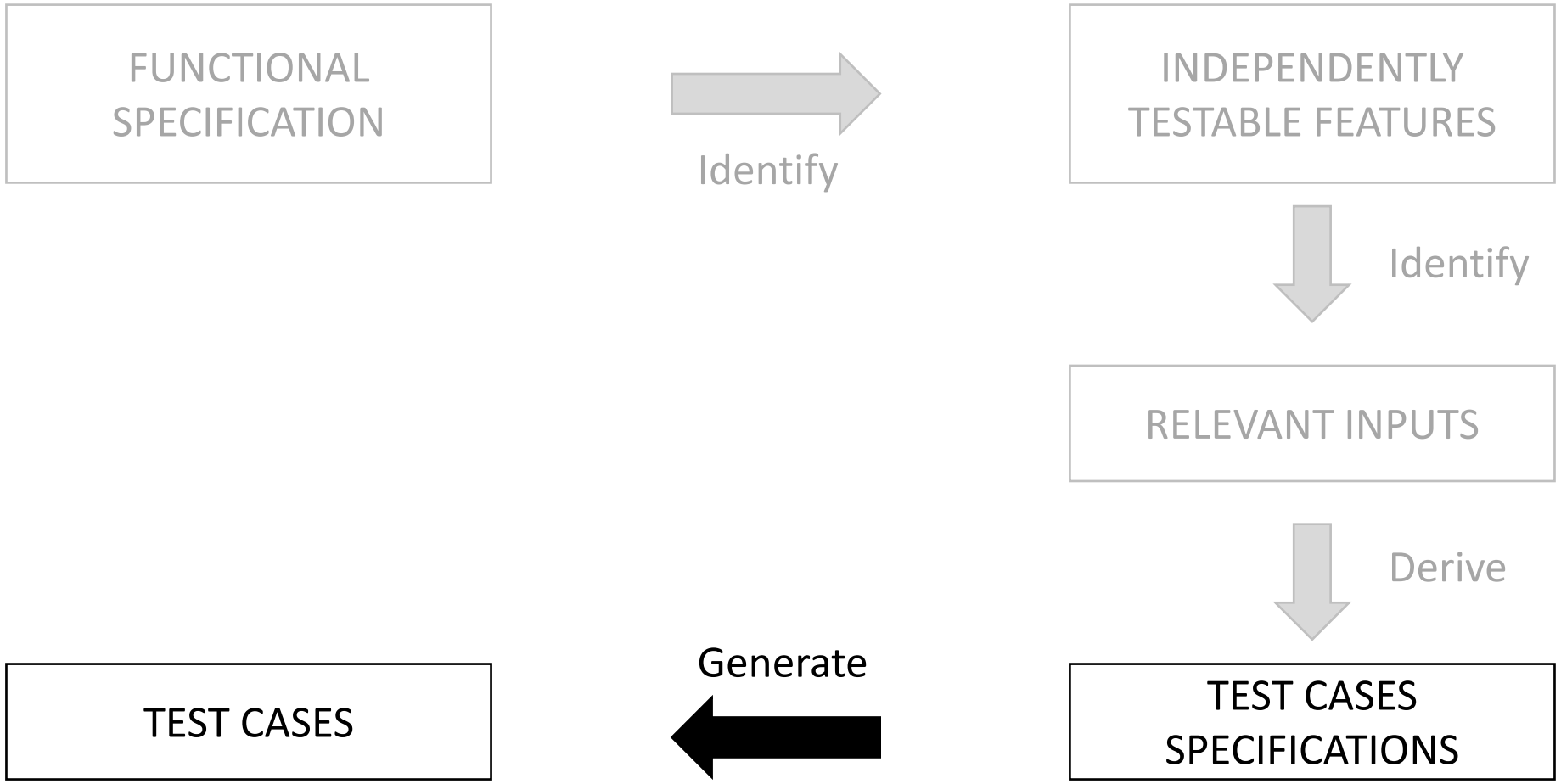


- Str with length = Size- 1
- Str with length = Size
- …

Test Case Specifications: (combine input values)

- ~~Size = -1, Str with length = -2~~
- ~~Size = -1, Str with length = -1~~
- Size = 1, Str with length = 0
- Size = 1, Str with length = 1
- …

A systematic Functional-Testing Approach



Slide adapted from Alessandro Orso

A Specific Functional Testing Black-Box Approach
The Category-Partition Method

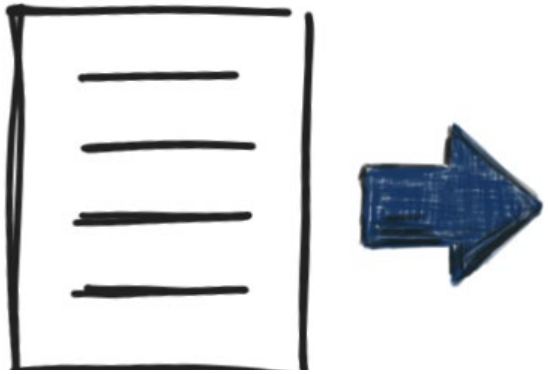
[Ostrand & Balcer, CACM, June 1988]



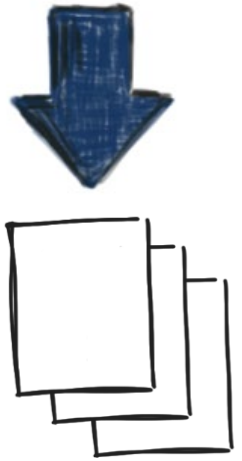
Specification

Test Cases

The Category-Partition Method



1. Identify independently testable features
2. Identify Categories
3. Partition Categories into choices
4. Identify constraints among choices
5. Produce/Evaluate test case specifications
6. Generate test cases from test case specifications



Test Cases

Identify Categories

Characteristics of each input element

split (string Str, int Size)

Input Str

- Length
- Content

Input Size

- value

Partition Categories into choices

Interesting cases (subdomains) – boundary values

split (string Str, int Size)

Input Str

- Length
  - 0
  - Size-1
- Content
  - Only Spaces
  - Special characters

Input Size

- Value
  - 0
  - >0
  - <0
  - MAXINT
  - …

Identify Constraints among choices

To Eliminate meaningless combinations & To reduce number of test cases

Three types: PROPERTY---- IF, ERROR, SINGLE

Input Str

- Length
  - 0 PROPERTY zerovalue
- Content
  - Special characters If !zerovalue

Input Size

- Value
  - <0 ERROR
  - MAXINT SINGLE

Produce And Evaluate Test Case Specifications

Can be automated

Produces test frames

Example (specify the characteristic of the inputs for that test)

Test frame #45
- Input Str
  - length: size -1
  - content: special characters
- Input Size
  - value: >0

Produce and evaluate test case specification
-how many test frames?
-add additional constraints to reduce the number if required

Generate Test Cases from Test Case Specification

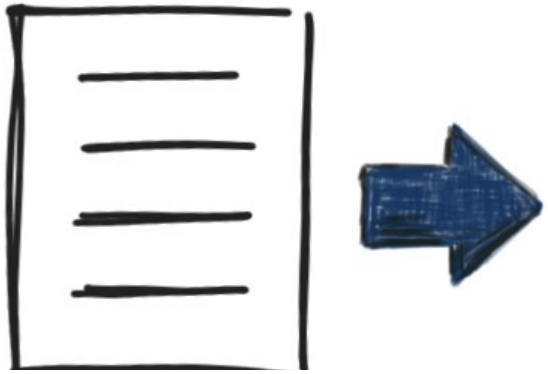
Simple Instantiation of frames

Final result: Set of concrete tests

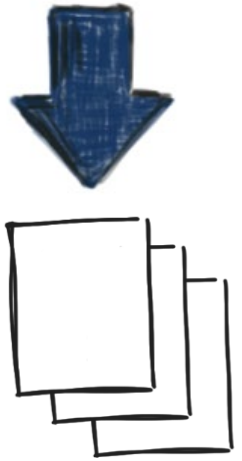
Example (specify the characteristic of the inputs for that test)

Test case #45
Str = “ABCC!\n\t”
Size = 10

The Category-Partition Method



1. Identify independently testable features
2. Identify Categories
3. Partition Categories into choices
4. Identify constraints among choices
5. Produce/Evaluate test case specifications
6. Generate test cases from test case specifications



Test Cases

Category Partition DEMO TIME

- Use category partition to generate test frames from a specification file (with categories, partitions, and constraints)
- Tool called TSLgenerator is used: Developed by team at UC Irvine, Oregon State, and Georgia Tech
- Download from: https://github.com/alexorso/tslgenerator/tree/master/Binaries
- run the code from command prompt: **./TSLgenerator-win8.exe**
- For help: **./TSLgenerator-win8.exe –manpage**
- To get number of test cases and write the test frames against your specification file: **./TSLgenerator-win8.exe -c *filename***

Can AI Generate Good Black-Box Tests?

Research question: Can an LLM systematically derive correct test cases from a complex functional specification?

Compare two approaches:

- **Approach A**: General test-generation prompt.
- **Approach B**: Structured category-partition prompt.

Evaluate:

- Completeness of input categories and boundaries
- Correctness of expected results
- Handling of constraints and invalid combinations
- Redundancy and manual corrections
- Time required

Experiment: Discount and Checkout System

A checkout system calculates the final order total using:

- Order subtotal
- Customer type: Regular, Premium, Student
- Promotional code: SAVE10, FLAT20, or absent
- Membership discounts and promotional eligibility
- Discount limits and shipping thresholds
- Input validation and rounding rules

Challenge: Identify meaningful partitions, constraints, and boundary cases.

https://chatgpt.com/share/6ae105f8-f9c0-83ea-b277-392196e652cc

Human Review of AI-Generated Tests

Evaluation	General Prompt	Category-Partition Prompt
Volume	185 scenarios	20 test frames + 35 concrete tests
Useful test	TC-174: Reordering request fields should not change the result	T-28: $10.05 with `SAVE10` tests half-cent rounding
Additional coverage	TC-173: Repeated calculation should produce the same result	T-08/T-09: Test just below and at a promotional threshold
Limitations	Many overlapping tests; some extend beyond the specified functionality	Introduces assumed discount rates, stacking rules, and shipping fees
Human review needed	Prioritize relevant, nonredundant tests	Verify assumptions, constraints, and expected results

What should a human reviewer catch?

Issue	Example from AI Output
Incorrect test design	General TC-017: Tests whitespace handling without whitespace in the input
Scope expansion	General TC-165: Tests concurrent order requests
Unconfirmed constraints	Category-Partition TF-14: TF-14 proposes combined discounts exceeding the cap without established stacking rules
Invented expected results	Category-Partition T-11: Calculates a $12 total using using illustrative discount and shipping rules

Takeaway: Human review must verify test relevance, feasibility, input correctness, and expected results.